

JSP – EXAM REVISION NOTES

Advanced Java | Short notes for quick reading and learning

How to study: Learn the definitions, life-cycle order, directive syntax, and differences. Practice writing the short examples.

1. Introduction to JSP

JSP (JavaServer Pages) is a server-side technology used to create dynamic web pages. It allows HTML to be written normally with small pieces of Java or JSP tags. The JSP container automatically converts a JSP page into a Servlet.

Why JSP? Writing HTML using many `out.println()` statements in a Servlet is difficult. JSP makes page design easier because HTML is the main content.

2. JSP Architecture

When a client requests a `.jsp` page, the JSP engine (for example, Jasper in Tomcat) performs these steps:

Step	Process
1. Request	Client requests the JSP page.
2. Translation	JSP is converted into a Java Servlet source file (.java).
3. Compilation	The Java source is compiled into a .class file.
4. Execution	The container loads and runs the class and sends HTML to the browser.

Remember: Translation and compilation usually happen on the first request or after the JSP changes. Later requests use the compiled Servlet.

3. JSP Life Cycle

The JSP life cycle is managed by the container:

- **Translation:** JSP source is converted into Servlet source.
- **Compilation:** Servlet source is compiled.
- **jspInit():** called once when the JSP is initialized.
- **_jspService():** called for every request; generated automatically.
- **jspDestroy():** called once before the JSP is removed from service.

Order to memorize: Translation → Compilation → `jspInit()` → `_jspService()` → `jspDestroy()`.

4. JSP Directives

Directives give instructions to the JSP engine. They configure the page and do not directly produce output.

Directive	Purpose	Example
page	Sets page settings such as imports, content type, and error page.	<code><%@ page import="java.util.*" contentType="text/html" %></code>
include	Includes another file at translation time (static include).	<code><%@ include file="header.jsp" %></code>
taglib	Declares a tag library such as JSTL.	<code><%@ taglib uri="..." prefix="c" %></code>

Static vs dynamic include: `<%@ include %>` includes content during translation. `<jsp:include>` includes output at request time.

5. Scriptlets, Expressions and Declarations

These are the three ways to embed Java code in JSP:

Element	Syntax	Purpose
Scriptlet	<% Java statements; %>	Java statements, loops, and conditions. Runs inside <code>_jspService()</code> .
Expression	<%= expression %>	Prints the value of an expression. No semicolon.
Declaration	<%! declaration %>	Declares class-level variables or methods.

Example:

```
<%! int square(int n) { return n*n; } %>
<% String name = "Asha"; %>
Welcome <%= name %>
Square: <%= square(5) %>
```

Declaration creates a method, scriptlet creates a variable, and expression displays values.

6. JSP Standard Actions

Standard actions are built-in JSP tags. They start with `jsp:` and perform common tasks.

- **<jsp:include page="..." />** – includes another resource's output at request time.
- **<jsp:forward page="..." />** – forwards the request to another resource.
- **<jsp:useBean ... />** – creates or finds a JavaBean.
- **<jsp:setProperty ... />** – sets a bean property.
- **<jsp:getProperty ... />** – retrieves and prints a bean property.

7. JavaBean and Bean Scopes

A JavaBean is a reusable Java object with properties, usually private fields and public getter/setter methods. JSP standard actions can create and access a bean.

Example: `<jsp:useBean id="student" class="StudentBean" scope="request" />`

Scope	Availability
page	Only on the current JSP page (default).
request	For the current request; remains available through a forward.
session	Across the user's session, until timeout or logout.
application	Across the entire web application; shared by users.

8. Expression Language (EL)

EL is a short syntax used to access data in JSP without writing Java code. It is written as `${expression}`.

Examples:

- `${param.username}` – reads a request parameter.
- `${student.name}` – reads the bean property (calls `getName()`).
- `${10 + 20}` – arithmetic expression.
- `${marks >= 40 ? "Pass" : "Fail"}` – conditional expression.

Common implicit objects: `param` (one request parameter), `paramValues` (multiple values), `header` (request header), `cookie` (request cookies), `pageScope`, `requestScope`, `sessionScope`, and `applicationScope` (attributes in each scope).

9. EL vs Scriptlets

Scriptlet	EL
Uses Java code mixed with HTML.	Short and HTML-friendly syntax.
Can make pages harder to read and maintain.	Cleaner for displaying data.
Complex logic may be written here, though it is better placed in a Servlet.	Recommended for displaying data; commonly used with JSTL.

JSTL & JavaBeans

ADVANCED JAVA | EXAM REVISION NOTES

1. JSTL Core Tags

JSTL stands for **JavaServer Pages Standard Tag Library**. It is a collection of predefined tags used in JSP for displaying data, conditions, loops and database operations. It makes JSP easier to write by reducing repeated Java code (scriptlets).

Why do we use JSTL?

- To reduce Java code in JSP.
- To perform conditions like if and else.
- To use loops like for.
- To display data dynamically.
- To make JSP pages easier to read and maintain.

JSTL is a collection of custom tags that encapsulate commonly used JSP functionality. It reduces scriptlet usage, improves readability, and provides built-in tags for iteration, conditional checks and data formatting.

JSTL has five categories: Core, Formatting, Internationalization and Messaging, Function, and SQL.

1.1 Core Tags

Core tags provide general-purpose functionality for conditions, loops, variables and URL handling. Add this directive at the top of the JSP page:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

No.	Tag	Description	Example
1	<c:set>	Sets the value of a variable.	<c:set var="name" value="Geeks"/>
2	<c:remove>	Removes a variable from a specific scope.	<c:remove var="name" scope="session"/>
3	<c:out>	Displays data on the page.	<c:out value="{username}"/>
4	<c:forEach>	Iterates over a collection or array.	<c:forEach var="i" begin="1" end="5">\${i} </c:forEach>
5	<c:import>	Includes the content of a URL.	<c:import url="header.jsp"/>

1.2 Formatting Tags

Used to format numbers, percentages and currency. Add this directive:

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

1. <fmt:formatNumber> — Formats numerical values, including separators, currency symbols and decimal separators.

Example:

```
<c:set var="numericValue" value="12345.6789"/>
<h3>Original Numeric Value: ${numericValue}</h3>
<fmt:formatNumber
  value="{numericValue}"
  type="currency"
  pattern="##,###.##"
  maxIntegerDigits="5"
  minIntegerDigits="3"
  maxFractionDigits="2"
  minFractionDigits="1"
  groupingUsed="true"
  var="formattedNumber"
```

```
scope="request"/>
<h3>Formatted Number: ${formattedNumber}</h3>
```

Output

Original Numeric Value: 12345.6789

Formatted Number: 12,345.68

2. **<fmt:formatDate>** — Formats the given date and time for display.
3. **<fmt:parseDate>** — Parses a date or time string into a standard Java Date object.
4. **<fmt:timeZone>** — Creates a localized time-zone component wrapper.
5. **<fmt:setTimeZone>** — Stores a specific time zone in a scope configuration variable.

1.3 Internationalization and Messaging Tags

These tags manage multilingual support and display locale-specific text messages.

- **<fmt:setLocale>** — Overrides the client's locale configuration for the page.
- **<fmt:bundle>** — Loads resources for message tags.
- **<fmt:message>** — Fetches and displays a localized key-value message.

1.4 Function Tags

Function tags provide various string functions. Add this directive:

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>
```

1. **fn:contains()** — Checks whether a string is contained inside another string.

```
<c:set var="name" value="GeeksforGeeks"/>
${fn:contains(name, 'Geeks')}
```

2. **fn:endsWith()** — Checks whether a string ends with a given suffix.

Example: `fn:endsWith(name, 'Geeks')`

3. **fn:escapeXml()** — Escapes characters that can be interpreted as XML.

```
<c:set var="name" value="&lt;abc&gt;Geeks for Geeks&lt;/abc&gt;" />
Without escapeXml: ${name}
With escapeXml: ${fn:escapeXml(name)}
```

4. **fn:split()** — Splits a string into a string array based on a separator.

2. JavaBeans / POJO

A **JavaBean** is a reusable Java class used to store and access data. It contains private variables and public getter and setter methods. It must have a public no-argument constructor. JavaBeans are commonly used in JSP to transfer data between Java classes and web pages.

Features of a JavaBean

- Private properties (variables).
- Public getter and setter methods.
- Public no-argument constructor.
- Serializable.
- Reusable and easy to maintain.

A JavaBean is a Java class also described in the notes as a POJO (Plain Old Java Object).

Example: StudentBean

```
public class StudentBean implements java.io.Serializable {
    private String firstName = null;
    private String lastName = null;

    public StudentBean() { }

    public String getFirstName() {
        return firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}
```

Calling a JavaBean in JSP

Use `<jsp:useBean>` to create or locate the bean, `<jsp:setProperty>` to set a property, and `<jsp:getProperty>` to read a property.

```
<jsp:useBean id="student" class="studentsBean"/>
<jsp:setProperty name="student" property="firstName" value="Bhavesh"/>
<jsp:getProperty name="student" property="firstName"/>
```

3. Java Persistence API (JPA)

The source notes list the following components:

- **Configuration:** Used to configure Hibernate and connect to the database.
- **Session Factory:** Creates and manages Session objects.
- **Session Object:** Used to perform database operations.
- **Transaction Object:** Manages database transactions and ensures data consistency.
- **Query Object:** Used to execute HQL or SQL queries.
- **Criteria Object:** Used to create database queries programmatically.

Quick revision: Remember the JSTL directives and tag purposes, the JavaBean features, and the StudentBean example.